

LAB EXAM MODEL QUESTIONS

1. Develop a Python application that allows a user to manage employee performance records through a graphical user interface. Design a class to represent an employee with attributes such as name, ID, and performance scores in multiple tasks. Include methods to compute total score, average performance, and performance rating. The application should display individual employee reports and generate a ranked list of employees based on their performance scores. Ensure proper input validation and store the final ranked list in a text file. A school maintains student performance data in multiple CSV files. Develop a Python program to combine these files, analyze subject-wise performance, identify top-performing students, and present the results using appropriate graphical visualizations.

The program integrates OOP, GUI, file handling, Pandas, and Matplotlib into a single application for performance management and analysis.

1. Created **Employee class (OOP)**
2. Designed **GUI using Tkinter**
3. Stored employee data & computed performance
4. Implemented **ranking + file storage**
5. Used **Pandas to merge CSV files**
6. Performed **analysis (groupby, sorting)**
7. Displayed results using **Matplotlib visualization**

```
# =====
```

```
# COMBINED PROGRAM: EMPLOYEE + STUDENT ANALYSIS
```

```
# =====
```

```
import tkinter as tk
```

```
from tkinter import messagebox
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
# =====  
  
# PART 1: EMPLOYEE CLASS  
  
# =====  
  
class Employee:  
    def __init__(self, name, emp_id, scores):  
        self.name = name  
        self.emp_id = emp_id  
        self.scores = scores  
  
    def total(self):  
        return sum(self.scores)  
  
    def average(self):  
        return self.total() / len(self.scores)  
  
    def rating(self):  
        avg = self.average()  
        if avg >= 80:  
            return "Excellent"  
        elif avg >= 60:  
            return "Good"  
        else:  
            return "Average"
```

```
employees = []
```

```
# =====
```

```
# GUI FUNCTIONS
```

```
# =====
```

```
def add_employee():
```

```
    try:
```

```
        name = name_entry.get()
```

```
        emp_id = id_entry.get()
```

```
        scores = list(map(int, score_entry.get().split(",")))
```

```
        emp = Employee(name, emp_id, scores)
```

```
        employees.append(emp)
```

```
        messagebox.showinfo("Success", "Employee Added")
```

```
    except:
```

```
        messagebox.showerror("Error", "Invalid input")
```

```
def show_report():
```

```
    output.delete("1.0", tk.END)
```

```
    for emp in employees:
```

```
        output.insert(tk.END,
```

```
        f"{emp.name} | Total: {emp.total()} | Avg: {emp.average():.2f} |  
        {emp.rating()}\n"  
    )
```

```
def rank_employees():
```

```
    ranked = sorted(employees, key=lambda x: x.total(), reverse=True)
```

```
    with open("employee_rank.txt", "w") as f:
```

```
        for i, emp in enumerate(ranked, 1):
```

```
            f.write(f"{i}. {emp.name} - {emp.total()}\n")
```

```
    messagebox.showinfo("Saved", "Ranking saved!")
```

```
# =====
```

```
# PART 2: STUDENT ANALYSIS FUNCTION
```

```
# =====
```

```
def analyze_students():
```

```
    try:
```

```
        file1 = file1_entry.get()
```

```
        file2 = file2_entry.get()
```

```
        data1 = pd.read_csv(file1)
```

```
        data2 = pd.read_csv(file2)
```

```
        data = pd.concat([data1, data2])
```

```
# Subject-wise avg
subject_avg = data.groupby("Subject")["Marks"].mean()

# Top students
top = data.sort_values(by="Marks", ascending=False).head(5)

result.delete("1.0", tk.END)
result.insert(tk.END, "Subject Average:\n")
result.insert(tk.END, str(subject_avg) + "\n\n")
result.insert(tk.END, "Top Students:\n")
result.insert(tk.END, str(top))

# Plot
plt.bar(subject_avg.index, subject_avg.values)
plt.title("Subject Performance")
plt.xlabel("Subject")
plt.ylabel("Average Marks")
plt.show()

except Exception as e:
    messagebox.showerror("Error", str(e))
```

```
# =====  
  
# GUI DESIGN  
  
# =====  
  
root = tk.Tk()  
root.title("Combined Application")  
  
  
# Employee Section  
tk.Label(root, text="EMPLOYEE SECTION").pack()  
  
  
name_entry = tk.Entry(root)  
name_entry.pack()  
name_entry.insert(0, "Name")  
  
  
id_entry = tk.Entry(root)  
id_entry.pack()  
id_entry.insert(0, "ID")  
  
  
score_entry = tk.Entry(root)  
score_entry.pack()  
score_entry.insert(0, "Scores (e.g. 80,90,70)")  
  
  
tk.Button(root, text="Add Employee", command=add_employee).pack()  
tk.Button(root, text="Show Report", command=show_report).pack()  
tk.Button(root, text="Rank Employees", command=rank_employees).pack()  
  
  
output = tk.Text(root, height=5)
```

```
output.pack()
```

```
# Student Section
```

```
tk.Label(root, text="STUDENT ANALYSIS").pack()
```

```
file1_entry = tk.Entry(root)
```

```
file1_entry.pack()
```

```
file1_entry.insert(0, "file1.csv")
```

```
file2_entry = tk.Entry(root)
```

```
file2_entry.pack()
```

```
file2_entry.insert(0, "file2.csv")
```

```
tk.Button(root, text="Analyze Students",  
command=analyze_students).pack()
```

```
result = tk.Text(root, height=8)
```

```
result.pack()
```

```
root.mainloop()
```